



Curso de Python

Unidad 3. Programación Orientada a Objetos en Python.

ÍNDICE

Contenido

1. Introducción.....	3
1.1 Por qué surgió.....	3
1.2 Concretando	4
2. Clases y objetos en Python	6
2.1 Los constructores de clase.....	10
2.2 Encapsulación de las propiedades.....	11
2.3 Encapsulación de los métodos	12
2.4 Herencia	16
2.4.1 Sobreescritura de métodos	20
2.4.2 Herencia simple y múltiple	23
2.4.3 Función super().....	24
3 Resumen.....	38



Objetivos

En esta unidad aprenderás a:

- Qué es la programación orientada a objetos en general

- De qué va la herencia en la POO para reciclar tu código.
- Cómo podemos trabajar con la herencia y el polimorfismo
- Sacar partido del encapsulamiento y la abstracción

1. Introducción

La programación orientada a objetos (POO) es un paradigma de la programación en general, ya que se puede utilizar en la mayoría de los lenguajes de programación modernos. Aunque fue introducido en los años 70, no se hizo popular hasta un tiempo después.

Si buscamos la definición en la RAE, paradigma es **Teoría o conjunto de teorías cuyo núcleo central se acepta sin cuestionar y que suministra la base y modelo para resolver problemas y avanzar en el conocimiento**. Pero si buscamos otra de sus definiciones para entenderlo mejor, encontramos la primera, que pone Ejemplo o ejemplar. Así que podemos considerarlo como un ejemplo o modelo.

Por ejemplo, en arquitectura podría ser una maqueta o en la vida cotidiana un pensamiento o punto de vista que se asume como cierto.

1.1 Por qué surgió

Cada vez la complejidad de los programas iba aumentando con el paso de los años. Además, cuando programamos, hay gran cantidad de aplicaciones que realizan tareas muy similares y es importante no estar reinventando la rueda constantemente. La programación orientada a objetos intentaba resolver esto, para facilitar el reciclaje del código.

Una de las primeras formas de reutilizar el código fueron las funciones. Por ejemplo, cuando voy a calcular el máximo de varios números, lo más fácil era

realizar una función que hiciera esto y cada vez que me hacía falta, la llamaría con los parámetros necesarios.

Así pues, la POO es una herramienta perfecta que permite resolver cierto tipo de problemas de una forma mucho más sencilla, con menos código y más organizado. Agrupa bajo una clase un conjunto de variables y funciones, que pueden ser reutilizadas con características particulares creando objetos.

1.2 Concretando

Ahora vamos a pensar en un ejemplo concreto para tenerlo más claro.

Imagina un coche de manera abstracta, es decir, no pienses en una marca o modelo concreto.

Ese coche tiene unas propiedades (en POO atributos) que le podemos asignar: color, número de ruedas, número de puertas, material con que está hecho, etc. Además, el coche también tiene unas funciones (métodos en POO) que son las cosas que puede hacer un coche: arrancar, parar o bajar las ventanillas.

Así ya sabemos que una clase es algo abstracto. Por lo tanto habrá que concretar en algo en el mundo real.

Sigamos con el ejemplo del coche:

- Propiedades:
- Marca: Ford
- Tipo: monovolumen
- Modelo: C-max
- Largo: 4'20 metros
- Ancho: 2 metros
- Número de ruedas: 4
- Color: Gris
- Material: aluminio

Métodos:

- Arrancar
- Parar
- Girar
- Abrir maletero
- Abrir motor
- Abrir / Cerrar puerta

Todo esto que se ha listado se hace de una manera concreta en el lenguaje Python. Pero eso lo veremos más adelante.

Aunque no lo sepas, ya has usado los objetos y métodos sin darte cuenta. Como por ejemplo cuando trabajamos con cadenas de caracteres. Ya que tienen unas propiedades y unos métodos. Tenemos como ejemplo el método de obtener la longitud de una cadena (función len):

```
mensaje = 'Hola Mundo'
```

```
print( len(mensaje) )
```

Ahora ya que conocemos la POO de una manera genérica, pasemos a ver cómo se implementa esto en el lenguaje de programación Python



2. Clases y objetos en Python

En este punto del tema, aprenderemos a crear clases en Python y los objetos correspondientes a partir de estas clases.

Cabe recordar que un objeto tiene un estado, unas propiedades y un comportamiento.

Por ejemplo, vamos a crear la propiedad `largoCarroceria:400`. Así, cualquier objeto que creamos a partir de la clase `Coche`, va a tener un `largoCarroceria` de 400 centímetros. Aunque eso se puede modificar posteriormente.

Ahora, vamos a tu compilador de Python preferido y escribimos los siguiente:

```
class Coche():  
    largoCarroceria=400  
    anchoCarroceria=200  
    ruedas=4  
    color='gris'  
    enmarcha=False
```

De este modo, creamos una clase `Coche` con 5 propiedades. De todas maneras, más tarde podríamos añadir más propiedades o quitar alguna de las que tiene.

Lo siguiente es añadir un comportamiento a nuestra clase. Los métodos hacen que nuestras clases tengan un comportamiento concreto. Acuérdate de arrancar, frenar, girar, derrapar, etc

Para crear un método usamos la palabra `def`. Aunque escribe lo siguiente y a continuación te lo explico.

```
def arrancar(self):  
    pass
```

En este caso, el método lo hemos denominado `arrancar`. Quizá lo más complicado sea entender el parámetro `self`. Hace referencia al propio objeto perteneciente a la clase, es decir a la instancia de la clase o más claro a un ejemplo en concreto.

En otros lenguajes, el self es equivalente a this. pass es una palabra del código que no hace nada, pero es obligatoria ponerla si de momento nuestro método no va a contener ningún código. Es decir, no podemos dejar únicamente

```
def arrancar(self):
```

Porque al compilarlo nos daría error.

Así que, de momento, ya tenemos una clase con 5 propiedades y un método.

Lo siguientes es construir algún objeto a partir de esta clase que acabamos de crear. ¿Cómo hacemos esto? De la siguiente manera:

```
miCoche=Coche()
```

 Esto es lo que se llama instanciar una clase

Además, si quiero acceder a la propiedad del objeto, escribiré lo siguiente:

```
print(miCoche.largoCarroceria)
```

 y me devolverá 400

Prueba con

```
print("El coche tiene ",miCoche.ruedas," ruedas")
```

a ver qué hace el programa.

A la hora de probar el método arrancar, vamos a modificarlo para que “haga algo”. Al estar trabajando de manera teórica, lo que hará el método arrancar es poner la propiedad enmarcha a True. Entonces, el método quedará de la siguiente manera:

```
def arrancar(self):  
    self.enmarcha=True
```

Y para “llamar” a este método lo haremos así:

```
miCoche.arrancar()
```

Y para conocer en qué estado se encuentra el coche, añadimos el siguiente método:

```
def estado(self):
    if(self.enmarcha):
        return "El coche está en marcha"

    else:
        return "El coche está parado"
```

Y comprobamos el estado:

```
print(miCoche.estado())
```

Y lo que devuelve el programa al final es lo siguiente:

```
El largo del coche es, 400
El coche tiene 4 ruedas
El coche está en marcha
```

Vamos a crear un segundo objeto, algo que ya sabes hacer sin ayuda.

```
miCoche2=Coche()
```

Este objeto que acabamos de crear, tiene una serie de características comunes con el primer objeto. Puedes comprobarlo de la siguiente manera:

```
print("El largo del coche es, ",miCoche2.largoCarroceria)
print("El coche tiene ",miCoche2.ruedas," ruedas")
```

Cambia el código para que haga lo siguiente (en este caso no arrancamos el segundo coche):

```
El largo del coche es, 400
El coche tiene 4 ruedas
El coche está en marcha
-----Creamos el segundo objeto coche-----
El largo del coche es, 400
El coche tiene 4 ruedas
El coche está parado
```

Ahora vamos a modificar el método arrancar, para que cuando arranque nos muestre también el estado en que se encuentra:

```
def arrancar(self, arrancamos):
    self.enmarcha=arrancamos
    if(self.enmarcha):
        return "El coche está en marcha"
    else:
        return "El coche está parado"
```

Modificamos también el método estado, que imprimirá directamente el estado del coche, por lo que cuando lo probemos ya no hará falta colocarlo dentro de un print.

Ahora, reorganizando el código que teníamos anteriormente para que no muestre información superflua por duplicado, copia el siguiente código para probar todo junto:

```
miCoche=Coche()

print(miCoche.arrancar(True))
miCoche.estado()
print("-----Creamos el segundo objeto coche-----")

miCoche2=Coche()

print(miCoche2.arrancar(False))
miCoche2.estado()
```

Y el resultado es el siguiente:

```
El coche está en marcha
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
-----Creamos el segundo objeto coche-----
El coche está parado
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
```

Es lógico que las propiedades que son comunes a todos los objetos que creemos sean iguales. Es lo que se denomina estado inicial.

```
class Coche():  
    largoCarroceria=400  
    anchoCarroceria=200  
    ruedas=4  
    color='gris'  
    enmarcha=False
```

2.1 Los constructores de clase

Pero cómo podemos especificar estas propiedades iniciales en el mundo de las clases en Python. Es lo que denominamos constructor. Así especificamos cuál va a ser el estado inicial de los objetos que creemos.

En este caso es un ejemplo sencillo, con unas pocas propiedades. Pero en el caso de clases con cientos de líneas, será imprescindible especificar un constructor en el que se pueda apreciar de forma clara el estado que van a tener los objetos.

Vamos a ver cómo podemos utilizar ese constructor. Para ello utilizaremos la palabra reservada `init`, precedida por dos guiones bajos y seguida por otros dos:

```
def __init__(self):
```

Ahora te pongo el código para que puedas probarlo.

```
class Coche():  
  
    def __init__(self):  
  
        self.largoCarroceria=400  
        self.anchoCarroceria=200  
        self.ruedas=4  
        self.color='gris'  
        self.enmarcha=False
```

A partir de ese momento, cualquier objeto que creemos, va a tener esas propiedades. El código funciona igual que antes de hacer los cambios del `init`, pero lo que hace es añadir a las propiedades un estado inicial.

Ahora te invito a que hagas el siguiente cambio para asignarle al segundo coche 2 ruedas. En nuestro código que estamos realizando, añade la siguiente línea antes de la última. Es decir, el código debería quedar de la siguiente manera:

```
miCoche2.ruedas=3
miCoche2.estado()
```

Y si lo ejecutamos debería mostrar lo siguiente:

```
El coche está en marcha
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
-----Creamos el segundo objeto coche-----
El coche está parado
El coche tiene 3 ruedas. Un ancho de 200 y un largo de 400
```

Y esto es algo que no deberíamos permitir, que se pudiera cambiar el número de ruedas, ya que perdería la esencia de lo que es un coche. Lo estudiamos en el siguiente apartado.

2.2 Encapsulación de las propiedades

En qué consiste este término. Lo que hacemos es proteger una propiedad para que no se pueda modificar desde fuera de la clase. Hacemos esto para añadir más protección a nuestra clase. Aunque a veces puede que no sea necesario, pero conozcamos cómo se hace. Para indicar la encapsulación, debemos preceder la propiedad de dos guiones bajos

```
self.__ruedas=4
```

De este modo la propiedad ruedas no será accesible desde el exterior de la clase. Por el contrario, sí podremos acceder a ella desde dentro de la propia clase.

Por ejemplo, sí queremos acceder a la propiedad, deberemos escribirla del mismo modo que la hemos definido.

```
def estado(self):
    print("El coche tiene ",self.__ruedas, " ruedas. Un ancho de ",self.anchoCarroceria, " y un largo de ",
          self.largoCarroceria)
```

Así, aunque modifiquemos en el programa el número de ruedas, nos seguirá diciendo que tiene 4 ruedas.

Puedes pensar que si modificamos la propiedad escribiéndola como la hemos definido podemos acceder a esa propiedad y modificarla.

Pero como puedes comprobar, aunque escribamos:

`miCoche2.__ruedas=3`, nos seguirá diciendo que nuestro coche tiene 4 ruedas.

Modificamos la clase para que todas las propiedades no sean modificables desde fuera:

```
class Coche():

    def __init__(self):

        self.__largoCarroceria=400
        self.__anchoCarroceria=200
        self.__ruedas=4
        self.__color='gris'
        self.__enmarcha=False
```

Y recuerda que para acceder a ellas desde la propia clase escribiremos los guiones bajos delante.

2.3 Encapsulación de los métodos

Para el siguiente apartado seguiremos trabajando con el archivo de la clase coche que estábamos utilizando.

Al igual que pasaba con las propiedades, encapsular un método quiere decir que sólo estará accesible desde dentro de la clase y no desde fuera de ella.

Seguro que hasta el momento has aprendido la sintaxis de cómo utilizar el encapsulamiento, pero quizá te estés preguntando qué utilidad pueda tener esto. Qué ventajas puedas aprovechar de que un método sólo sea accesible desde dentro de la clase y no desde el exterior. Todo eso lo aprenderás a lo largo de este apartado.



Recuerda que hasta el momento hemos creado la clase coche, con una serie de propiedades que solo son accesibles desde el interior de la clase.

Vamos a simular el chequeo que realiza un coche cuando pulsamos el botón de arrancar o giramos la llave (según como sea el coche de moderno). Lo único que hará el método es mostrar el estado de la gasolina, el aceite etc, pero sin complicarnos mucho. Ya que, aunque debería comprobar el valor de las variables, únicamente las mostrará por pantalla.

```
def auto_check(self):  
    print("Vamos a realizar el chequeo")  
  
    self.gasolina="ok"  
    self.aceite="ok"  
    self.puertas="cerradas"  
  
    if (self.gasolina=="ok" and self.aceite=="ok" and self.puertas=="cerradas"):  
        return True  
    else:  
        return False
```

Para utilizar este método, lo haremos justo antes de llamar al método arrancar, como ocurre en la vida real cuando arrancamos un coche.

Te muestro el siguiente código y a continuación va la explicación

```
def arrancar(self, arrancamos):  
    self.__enmarcha=arrancamos  
  
    if (self.__enmarcha):  
        check=self.autocheck()  
  
    if(self.__enmarcha and check):  
        return "El coche está en marcha"  
  
    elif(self.__enmarcha and check=False):  
        return "Chequeo no superado. Arranque no posible"  
  
    else:  
        return "El coche está parado"
```

Cuando vamos a arrancar el coche hacemos el chequeo (en nuestro ejemplo siempre va a ser positivo). Entonces, si el coche está en marcha y el chequeo ha

sido ok, mostrará el mensaje El coche está en marcha. Al contrario, si el chequeo ha sido incorrecto e intentamos arrancar, nos dirá que El cheque no se ha superado y es imposible arrancar. Por último, si el coche no arranca querrá decir que está parado.

La instrucción elif se utiliza para comprobar múltiples condiciones si una condición es falsa. Es similar a la sentencia if-else pero la diferencia es que elif evalúa múltiples condiciones, a diferencia de else.

Si lo ejecutas, el resultado debe ser éste:

```
Vamos a realizar el chequeo
El coche está en marcha
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
-----Creamos el segundo objeto coche-----
El coche está parado
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
```

La diferencia entre el primer Coche 1 y el Coche 2, es que en el primero, al llamar al método arrancar le pasábamos el parámetro True, mientras que en el segundo le pasábamos el parámetro arrancamos a False, por lo que en este segundo caso no llega a hacer el chequeo y no arranca.

Para seguir haciendo pruebas con nuestro programa, vamos a poner que el estado del aceite es incorrecto. De la siguiente forma:

```
self.aceite="nivel incorrecto"
```

Si volvemos a probar el programa, al estar mal el nivel del aceite no pasará el chequeo, por lo que deberá notificárnoslo.

```
Vamos a realizar el chequeo
Chequeo no superado. Arranque no posible
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
-----Creamos el segundo objeto coche-----
El coche está parado
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
```

Podríamos comprobar por qué no se ha superado el chequeo. Te dejo que lo realices por tu cuenta.

Si recuerdas, en este apartado estamos estudiando la encapsulación de los métodos. De momento, el que hemos creado del `auto_check(self)` es accesible desde fuera de la clase. Para probarlo, escribe las últimas instrucciones siguientes.

Pero antes escribe que el nivel del aceite es correcto con `self.aceite="ok"`

Incluso el Coche 2 podría hacer un chequeo estando parado, algo que no tendría sentido.

```
miCoche=Coche()
print(miCoche.arrancar(True))
miCoche.estado()
print(miCoche.auto_check())
print("-----Creamos el segundo objeto coche-----")
miCoche2=Coche()
print(miCoche2.arrancar(False))
miCoche2.estado()
print(miCoche2.auto_check())
```

Y éste es el resultado que obtenemos:

```
El coche está en marcha
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
Vamos a realizar el chequeo
True
-----Creamos el segundo objeto coche-----
El coche está parado
El coche tiene 4 ruedas. Un ancho de 200 y un largo de 400
Vamos a realizar el chequeo
True
```

Entonces, si queremos que no se pueda acceder a un método en el momento que queramos, lo lógico es encapsularlo. Así solo se podrá usar desde dentro de la clase. En el caso en el que estamos, el método de `auto_check` sólo podrá utilizarse dentro de la clase.

Para realizar esto, procederemos como hacíamos con las propiedades:

```
def __auto_check(self):
```

Sólo pongo la definición para no ser redundante. El resto del método sería igual. Y donde utilicemos el método, lo precederemos de dos guiones bajos también.

```
if (self.__enmarcha):  
    check=self.__auto_check()
```

Si somos un poco malos y pensamos lo siguiente: Pues llamo al método en mi programa anteponiéndole los dos guiones bajos. Pero Python va un paso por delante de nosotros y no nos deja ejecutar esa orden.

```
AttributeError: 'Coche' object has no attribute '__auto_check'
```

Por lo que habrá que borrar las entradas desde fuera.

Resumiendo este apartado, ni siempre va a ser necesario encapsular propiedades o métodos o no encapsularlos. Eso va a depender de las necesidades y a medida que vayamos cogiendo experiencia en Python. Seremos nosotros los que decidiremos la encapsulación o no de los métodos o propiedades.

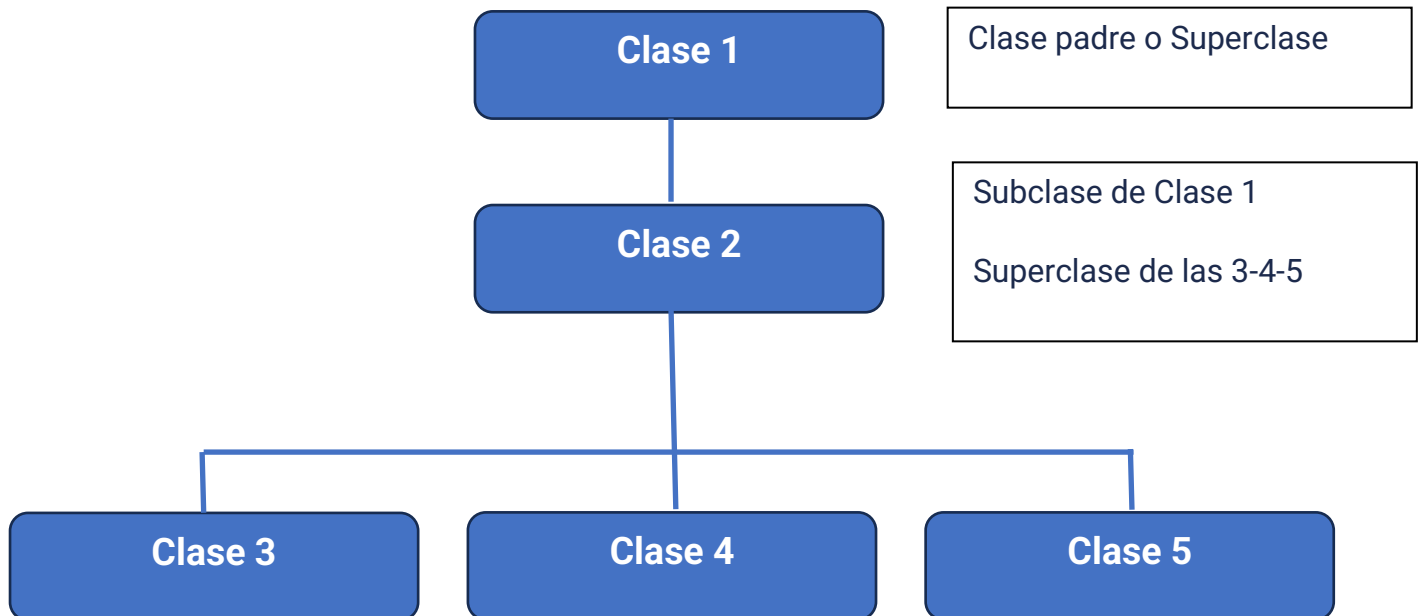
2.4 Herencia

Como en cualquier apartado, estudiaremos qué es la herencia, para qué sirve y cómo hacer que una clase pueda heredar de otra.

¿Qué es lo primero que se te viene a la cabeza cuando escuchas la palabra herencia? Quizá dinero, una casa, un coche. Pero en el caso de la programación, la herencia está más relacionada con la herencia física. Es decir, si tus padres son altos tú serás alto, o si tus padres tienen la piel oscura quizá tú también la tengas, o tus hijos. Todo esto habrá que trasladarlo a código de programación.

Dándote una primera pincelada para que lo entiendas, recuerda que nosotros creamos una clase coche en el punto anterior. Si yo ahora quiero crear la clase formula_uno, no debo partir de cero. Puedo coger las características que tenía la clase coche anteriormente diseñada y añadirle las propiedades necesarias que quizá no eran necesarias en la clase padre. Como podría ser aceleración. O métodos nuevos como cambiar_ruedas_en_carrera.

La herencia es algo que nosotros utilizaremos para poder reutilizar código y, como se suele decir, no estemos siempre reinventado la rueda.



Al igual que pensábamos cuando empezamos a trabajar con la POO, podemos pensar en cómo puedo yo utilizar la herencia.

Puedes pensar en un coche, un autobús, un camión, una moto ... Porque en programación, es casi más importante el análisis que la propia programación. Un buen análisis puede ahorrarnos un montón de horas de programación y de quebraderos de cabeza.



Siguiendo con nuestro ejemplo, lo primero que debemos pensar es qué características (acuérdate de las propiedades) tienen en común todos estos objetos:

- Marca
- Modelo

Para empezar, no está mal, pero sigamos.

Y además, qué comportamientos tienen en común nuestros diferentes objetos (métodos en nuestro caso):

- Arrancan
- Aceleran
- Frenan

¿Qué nos permite la herencia? Que no tenga que crear una clase para el coche, otra para el camión y otra para la moto. Lo que haremos será crear una clase (clase padre o superclase), con todos los elementos comunes que tengan entre los dos (ya sean propiedades o métodos), como ya hemos visto y que cada subclase añada lo necesario para poder trabajar. Por ejemplo, si un camión tiene una propiedad que sea el volumen de la carga, eso es algo que no va a tener una moto. Aunque tienen en común que los dos arrancan.

Pero pongámonos manos a la obra para aprender cómo trabajar la herencia con Python y ejecutamos nuestro editor preferido.

Copia el siguiente código para crear la clase:

```
class vehiculos():  
    def __init__(self,marca,modelo):  
        self.marca=marca  
        self.modelo=modelo  
        self.enmarcha=False  
        self.acelera=False  
        self.frena=False
```

Con este código se crea la clase vehículos, para lo cual hemos de pasarle como parámetros la marca y el modelo. Además, cuando lo construyamos añadiremos 3 propiedades más, asignándoles el valor False. Ya que, haciendo un símil, cuando se crea el vehículo, ni está en marcha ni acelerando ni frenando.

Lo siguiente será crear los comportamientos o métodos. Vamos a crear 4 métodos, de los cuales 3 están relacionados con las propiedades y otro que nos mostrará el estado de nuestro vehículo en cualquier momento.

```
def arrancar(self):
    self.enmarcha=True

def acelerar(self):
    self.acelera=True

def frenar(self):
    self.frena=True

def estado(self):
    print("Marca: ",self.marca,"\nModelo ",self.modelo, "\nEn marcha: ",
          self.enmarcha, "\nAcelerando ",self.acelera,"\nFrenando :",self.frena)
```

Lo siguiente será crear una clase que herede de la clase padre vehiculo. Para ello haremos lo siguiente:

```
class camion(vehiculos):
    pass
```

De esta manera creamos una clase llamada camion, que hereda todas las propiedades y métodos que tiene la clase vehículos. La palabra reservada pass la escribimos para rellenar de momento la clase. Ya que no podemos dejarla vacía. Más adelante completaremos la clase camion.

Ahora imaginemos que vamos a crear un objeto camion, concretando un poco más.

Como estamos heredando también el constructor, debemos pasarle la marca y el modelo cuando lo construyamos:

```
mi_camion=camion("volvo","FMX")
```

Ahora ya podemos heredar cualquiera de los métodos que hemos heredado:

```
mi_camion.estado()
```

Y nos devolverá el siguiente resultado:

```
Marca:  volvo
Modelo  FMX
En marcha:  False
Acelerando  False
Frenando :  False
```

2.4.1 Sobreescritura de métodos

Para el siguiente apartado, continuaremos con el ejemplo que hemos creado antes, el de los vehículos.

Como ya hemos comentado antes, habrá comportamientos o propiedades que no compartan los vehículos. En nuestro caso, nuestro camión puede descargar, lo que no puede hacer un coche o un vehículo anfibio.

```
class camion(vehiculos):  
    descargando=""  
    def descargar(self):  
        self.descargando="estoy descargando"
```

A nuestra clase camion le he añadido una propiedad llamada descargando, que en principio la dejamos vacía y un método llamado descargar, que lo que hace es colocar la cadena "estoy descargando" en dicha propiedad.

Te puedes preguntar, ¿Cuántos métodos tiene en este momento la clase camion? En este punto del programa tiene 6 métodos:

arrancar / acelerar / frenar / estado / descargar y no nos podemos olvidar del constructor.

Además, si yo quiero utilizar el método descargar que hemos creado, bastará con escribir

```
mi_camion.descargar()
```

Y ya estaríamos utilizándolo.

Pero en ningún momento tenemos constancia ni nos muestra que estamos descargando. Puedes pensar, pues modifico el método estado y le añado que muestre si estoy descargando de esta forma:

```
def estado(self):  
    print("Marca: ",self.marca,"\nModelo ",self.modelo, "\nEn marcha: ",  
          self.enmarcha, "\nAcelerando ",self.acelera,"\nFrenando :",self.frena,  
          self.descargando)
```

Pero estaríamos actuando de manera incorrecta, ya que así estaríamos diciendo que cualquier vehículo puede descargar. Lo cual en una moto, por ejemplo, sería algo incongruente.

¿Cómo se resuelve esto?

Pues que la clase camion cree su propio método estado para que informe, además de todo lo anteriormente, que está descargando o no. Es lo que se llama la sobreescripción de métodos.

Para ello, dentro de la clase camion crearemos un método estado, al cual le añadiremos que muestre también si estamos descargando o no.

```
def estado(self):  
    print("Marca: ",self.marca,"\nModelo ",self.modelo, "\nEn marcha: ",  
          self.enmarcha, "\nAcelerando ",self.acelera,"\nFrenando :",self.frena,  
          "\nDescarga: ", self.descargando)
```

Y como nos gusta reflexionar pensamos: si yo escribo lo siguiente:

```
mi_camion.estado()
```

¿A qué método llama Python? ¿Al de la clase vehiculo o al de la clase camion? En este caso, gracias a la sobrecarga, llamará al de la clase camion.

```
Marca:  volvo  
Modelo  FMX  
En marcha:  False  
Acelerando  False  
Frenando : False  
Descarga:  estoy descargando
```

Esto es así porque antes de comprobar el estado he llamado al método descargar. Si no lo hubiese hecho, mostraría lo siguiente:

```
Marca:  volvo  
Modelo  FMX  
En marcha:  False  
Acelerando  False  
Frenando : False  
Descarga:
```

Porque, por defecto, la variable descargando se crea vacía al principio.

Siguiendo con nuestras reflexiones, imaginemos que creamos una clase llamada volquete (es un camion pequeño que se utiliza en las obras) que hereda de nuestra clase camion.

```
class volquete(camion):  
    pass
```

Volquete ← camion ← vehiculo

En este caso, volquete heredará el método estado de la clase camion.

Siguiendo con nuestras pruebas, vamos a crear la clase moto.

```
class moto(vehiculos):  
    haciendo_caballito=""  
    def caballito(self):  
        self.haciendo_caballito=True
```

Le hemos añadido la propiedad caballito, porque es algo que no puede hacer un camion.

Y creamos una instancia:

```
mi_moto=moto("Yamaha","FZR")
```

Recuerda que como no hemos declarado ningún constructor, hereda el de la clase vehículos, para lo cual debíamos indicar la marca y el modelo.

Además, podemos llamar a los métodos que tenemos disponibles:

```
mi_moto.arrancar()  
mi_moto.acelerar()  
mi_moto.estado()
```

Y nos devolvería:

```
Marca: Yamaha  
Modelo FZR  
En marcha: True  
Acelerando True  
Frenando : False
```

2.4.2 Herencia simple y múltiple

Siguiendo con nuestros ejemplos, vamos a crear una clase que dé soporte a los vehículos eléctricos, lo cual parece que representa el futuro de la movilidad:

```
class v_electrico():
    def __init__(self):
        self.autonomia=500

    def cargar_bateria(self):
        self.cargando=True
```

Y si pensamos en una bicicleta eléctrica, llegamos a la conclusión que comparte características y comportamientos con la clase vehículos y con la clase v_electrico

Esto es lo que se denomina herencia múltiple, que es algo que soporta Python. Cuando heredamos de varias clases a la vez.

Para crear la clase bicicleta eléctrica bastaría con escribir:

```
class bici_electrica(v_electrico,vehiculos):
    pass
```

Y ahí surge otra duda. Cuando vamos a crear una instancia de la bicicleta, ¿qué constructor debemos usar? Porque el de v_electrico no tiene parámetros y al de vehículos debíamos pasarle la marca y el modelo.

Prueba lo siguiente:

```
mi_bici=bici_electrica("Pinarello","K3000")
```

Y al compilar nos contesta:

```
TypeError: v_electrico.__init__() takes 1 positional argument but 3 were given
```

Por lo que llegamos a la siguiente conclusión:

Debemos usar el constructor de la clase que ponemos en primer lugar. Por lo que en nuestro caso, para crear una instancia de la bicicleta eléctrica, únicamente deberemos escribir:

```
mi_bici=bici_electrica()
```

Si cambiáramos el orden de la herencia de la bici eléctrica sí que nos valdría la primera solución.

2.4.3 Función super()

Seguiremos con el ejemplo anterior, con la clase de bici_electrica. Recuerda que utilizaremos el constructor de la clase padre que aparece más a la izquierda.

```
class bici_electrica(v_electrico,vehiculos):  
    pass
```

Pero y si quisiésemos utilizar el constructor de la clase vehículos.

Hay dos formas de hacer esto. Uno menos ortodoxo que sería en irnos a la clase v_electrico y repetir la construcción que hicimos en vehículos. Dijéramos que sería copiar el constructor de la clase vehículos, donde sea necesario indicar la marca y el modelo.

Pero hay un método más elegante que nos permite hacer esto. Para ello utilizaremos la función super(). Donde la coloquemos va a llamar al método de la clase padre. Veamos un ejemplo.

Creemos un archivo nuevo, para no mezclarlo, de momento, con todo lo que tenemos programado ya.

Crea la siguiente clase:

```
class persona:
    def __init__(self,nombre,edad,poblacion):
        self.nombre=nombre
        self.edad=edad
        self.poblacion=poblacion

    def datos_persona(self):
        print("Nombre: ",self.nombre," Edad: ",self.edad," Poblacion: ",self.poblacion)
```

Tiene un constructor y un método que nos muestra los valores por pantalla.

Y a continuación creamos otra clase:

```
class empleado(persona):

    def __init__(self,sueldo,antigüedad):
        self.sueldo=sueldo
        self.antigüedad=antigüedad
```

Esta segunda clase hereda de la clase persona

Ahora ya estamos en condiciones en crear un objeto de la clase persona y llamar al método datos_persona

```
Pepe=persona("Pepe",50,"Albacete")
Pepe.datos_persona()
```

Y esto es lo que devuelve el programa

```
Nombre:  Pepe  Edad:  50  Poblacion:  Albacete
```

Ahora queremos construir un empleado:

Si lo hacemos así nos devolverá un error:

```
Pepe=empleado("Pepe",50,"Albacete")
```

Por lo que podemos pensar que debemos utilizar el constructor de empleado:

```
Pepe=empleado(2000,20)
```

Creamos a Pepe, que tendrá un sueldo de 2000€ y 20 años de antigüedad.

El problema llega cuando llamamos al método `datos_persona`.

```
print("Nombre: ",self.nombre," Edad: ",self.edad," Poblacion: ",self.poblacion)
AttributeError: 'empleado' object has no attribute 'nombre'
```

Porque no reconoce el atributo `nombre` ya que es el primero al que llega.

Esto lo solucionamos de la siguiente manera, a través de la función `super`

```
class empleado(persona):

    def __init__(self,sueldo,antigüedad):
        super().__init__("Pepe",50,"Albacete")
        self.sueldo=sueldo
        self.antigüedad=antigüedad
```

Lo que hace es, cuando entra al constructor de la clase `empleado`, llama al constructor de la clase padre, en este caso `persona`. Y así ya funciona correctamente.

Aunque habrá que depurar un poco nuestro código, ya que todos los empleados que creamos se llamarán `Pepe`, con 50 años y tendrán la residencia en `Albacete`. Lo cual no tiene sentido si queremos generalizar nuestro programa.

Para ello tenemos que hacer los siguientes cambios:

```
class empleado(persona):

    def __init__(self,sueldo,antigüedad,nombre_empleado,edad_empleado,poblacion_empleado):

        super().__init__(nombre_empleado,edad_empleado,poblacion_empleado)

        self.sueldo=sueldo
        self.antigüedad=antigüedad
```

Por lo hemos modificado el constructor del empleado y tenemos que cambiar esta línea:

```
Pepe=empleado(2000,20)
```

Ya que al nuevo constructor hemos de pasarle 5 parámetros

```
Pepe=empleado(2000,20,"Pepe",50,"Albacete")  
  
Pepe.datos_persona()
```

El método `datos_persona` nos muestra ciertos datos, pero no muestra ni el sueldo ni la antigüedad. Por lo que habrá que sobrescribir el método `datos_persona` de la clase `persona` para que muestre todo. Éste será el nuevo método en la clase `empleado`:

```
def datos_persona(self):  
  
    super().datos_persona()  
  
    print("Salario: ",self.sueldo,"Antigüedad: ",self.antigüedad)
```

Y así ya mostraría todo:

```
Nombre:  Pepe  Edad:  50  Poblacion:  Albacete  
Salario:  2000  Antigüedad:  20
```

Pero recuerda que todo esto puede ir complicándose al utilizar muchas clases, herencia, métodos ...

Para aclarar un poco todo esto, en el mundo de la herencia suele utilizarse lo que se llama el **principio de sustitución**. Consisten en aplicar los términos:

“es siempre un / una”

Cuando tenemos herencia, un objeto de la subclase siempre será un objeto de la superclase. En nuestro caso, un `Empleado` es siempre una `persona`. Pero al revés no siempre ocurre esto. Puede haber personas que no tengan empleo, pero al contrario sí como hemos dicho.

Todo esto lo estudiamos en el siguiente apartado.

2.4.4 Función isinstance()

Esta función nos ayuda cuando aumente mucho la complejidad de nuestros programas. Nadie dijo que la programación fuera sencilla.

`isinstance()` nos devuelve únicamente `True` o `False`. Además, se le han de pasar dos parámetros.

Pero probémosla para conocerla, escribiendo lo siguiente:

```
print(isinstance(Pepe, empleado))
```

Y nos devolverá `True`.

Pero, ¿qué pasa si intento comprobar si Pepe pertenece a la clase `persona`? Debido al principio de sustitución que hemos comentado en el apartado anterior, debería devolvernos `True`. Y es lo que ocurre.

```
print(isinstance(Pepe, persona))
```

Y si queremos complicarlo todo un poco más, podemos crear el objeto `Manolo` de la clase `persona` y preguntarle si es un empleado. En cuyo caso no debe devolver `False`.

Ahora volvemos a nuestro ejemplo de los vehículos y la bici eléctrica (no te he dicho que lo cerrarás).

Queremos conseguir que una bici eléctrica tenga su marca y su modelo.

Para ello, primero modificamos el constructor de `v_electrico`, llamando al constructor de `vehiculos`

```
class v_electrico(vehiculos):  
    def __init__(self, marca, modelo):  
        super().__init__(marca, modelo)  
        self.autonomia=500
```

Y también hemos de modificar la construcción de una bici eléctrica.

```
mi_bici=bici_electrica("Pinarello","ebike")
```

Te dejo como tarea que añadas a la clase de la bici eléctrica un método para que nos muestre toda su información

2.5 Polimorfismo

Si buscamos el término en la RAE, encontramos que polimorfismo es Cualidad de polimorfo. Y por si esto no queda claro, buscamos polimorfo: Que tiene o puede tener distintas formas.

En el mundo de la programación, y más en concreto en la Programación Orientada a Objetos, un objeto presenta polimorfismo cuando dependiendo del contexto se comporta de una manera u otra.

Para entenderlo más claro, lo veremos a través de un ejemplo. En el caso que estamos trabajando, un mismo objeto puede cambiar de forma, pasar de ser un camión a una moto o de una moto a un coche. Se cambiaría de comportamiento según el contexto.

Python es un lenguaje de programación de tipado dinámico, no necesitamos declarar de qué tipo es un objeto cuando vamos a trabajar con él. Algo que no ocurre en otros lenguajes.

Nos ponemos manos a la obra creando un nuevo archivo.

Dentro de ese archivo nuevo (yo sigo utilizando el editor Thonny), hemos de crear las siguientes clases:

```
class Coche():
    def moverse(self):
        print("me nuevo con 4 ruedas")

class Moto():
    def moverse(self):
        print("me nuevo con 2 ruedas")

class Camion():
    def moverse(self):
        print("me nuevo con 8 ruedas")
```

Y podemos probar creando un objeto de la clase moto y hacer que se mueva. Si no te acuerdas cómo hacerlo, te pongo yo el código.

```
mi_vehiculo=Moto()

mi_vehiculo.moverse()
```

Y si lo probamos, supongo que adivinas lo que nos devolverá:

```
me nuevo con 2 ruedas
```

Repetiremos creando un coche y haciendo que se mueva.

```
mi_vehiculo2=Coche()

mi_vehiculo2.moverse()
```

Ambas clases coinciden en un método o comportamiento llamado moverse.

Ya que tenemos una clase camión, la utilizamos.

No sé si te habrás dado cuenta hasta el momento, pero no es recomendable utilizar acentos, ñ y símbolo extraños a la hora de llamar una clase o método. No es que sea una falta de ortografía. Es lo que tiene que esto no lo hayan creado los españoles. Pero no es ningún problema.

Ahora ya tenemos a toda nuestra flota en marcha:

```
me muevo con 2 ruedas
me muevo con 4 ruedas
me muevo con 8 ruedas
```

Ahora pensemos que estamos programando un juego en Python (existe un módulo llamado Pygame que estudiaremos al final del curso) y en nuestro juego aparecen coches, motos, camiones y más tipos de vehículos.

Entonces, para moverlos tendríamos que proceder como acabamos de realizar. Pero Python dispone de algo que nos permitirá reducir el código y facilitar la redacción de nuestro programa.

Vamos a crear el siguiente método llamado moverVehiculo, al cual le pasaremos como parámetro un objeto:

```
def moverVehiculo(vehiculo):
    vehiculo.moverse()
```

Y creo un objeto moto y llamo al método moverVehiculo

```
mi_vehiculo=Moto()

moverVehiculo(mi_vehiculo)
```

Como ya adivinarás, éste es el resultado:

```
me muevo con 2 ruedas
```

En este caso, el polimorfismo aparece en este punto exactamente.

Según el parámetro que le pasemos, realizara el método moverse según el objeto que sea.

Si sustituyo la Moto() por Coche(), se transformará en un objeto de tipo Coche.

2.6 Abstracción

En Python podemos crear lo que se denominan clases abstractas. Y quizá este concepto nos suene algo extraño. Como antes, podemos tirar de la RAE y buscar el significado. Un nombre abstracto es un nombre que no designa una realidad material: p. ej., actitud, belleza, movimiento.

En Python, una clase abstracta presenta las siguientes características:

- No podemos crear objetos de ellas directamente
- Contienen, por lo menos, un método abstracto
- Se usan de base para crear subclases más específicas.

Métodos abstractos:

- Debemos sobrescribirlos en cada una de las subclases.

Podríamos crear una clase abstracta llamada herramienta, que podría servir de base para crear otras clases. Que contendría las características comunes de las herramientas. Pero yo no podría crear un objeto de la clase herramienta. Debería crear una subclase llamada, por ejemplo, clase sierra y de ahí crear (o instanciar) objetos.

Veremos esto a través de un ejemplo. Vamos a crear la clase abstracta personaje, y de ahí crearemos la clase mago y la clase guerrero.

La primera línea que contendrá es:

```
from abc import ABC, abstractmethod
```

Todavía no hemos visto las palabras clave `from` e `import`. En Python tenemos disponibles muchas librerías (conjunto de funciones, clases, etc) a nuestra disposición. Pero algunas están por defecto y otras debemos importarlas. Un ejemplo clásico es el módulo `math`. El cual utilizaremos para utilizar funciones matemáticas como `random` (números aleatorios) o calcular el factorial de un número.

En nuestro caso, el módulo abc se utiliza para crear clases abstractas. Si no utilizásemos ni from ni import no las tendríamos a nuestro alcance.

Copia el siguiente código y a continuación te explico lo que hace:

```
class Personaje(ABC):
    @abstractmethod
    def __init__(self, nombre):
        self.nombre=nombre
        self.nivel=0
        self.inventario=[]
        self.vida=100
```

Creamos una clase llamada personaje, definida como abstracta. En el init debemos pasarle el nombre y tiene 3 propiedades, nombre, nivel y el inventario. Que es un array.

Si intentamos crear un objeto de esta clase, nos debe dar error:

¿Te atreves a crear el personaje por tu cuenta? Si es así, no sigas leyéndolo y créalo a ver si te funciona. Si no, te lo digo yo

```
p1=Personaje("Merlin")
```



Y al ejecutarlo, nos da un error. No sé cuál es tu nivel de inglés (si estás en el mundo de la programación debería ser como mínimo de nivel medio), pero éste es el mensaje que nos devuelve:

```
TypeError: Can't instantiate abstract class Personaje with abstract method __init__
```

Que quiere decir que no podemos instanciar (es decir crear un objeto) de la clase abstracta Personaje.

Continuamos con nuestro código:

```
@abstractmethod
def atacar(self, objetivo):
    pass
```

Este método atacar deberá redefinirse en las clases que hereden, porque no es lo mismo que ataque un mago, que lo haga un guerrero. Recuerda que el método no podemos dejarlo vacío, por lo que ponemos la palabra reservada pass.

```
@abstractmethod
def estado(self):
    print("El personaje ",self.nombre," tiene un nivel de ",self.nivel)
```

También tendremos el método estado, que nos dirá en cada momento el nivel del personaje.

Crearemos el siguiente método:

```
def subir_nivel(self):
    self.nivel+=1
```

Este método no lo creamos abstracto ya que no hará falta redefinirlo. Todos los personajes aumentarán de nivel de la misma manera.

Por último solo nos queda crear el siguiente método, que quizá sea el que más te cueste de entender.

```
def ver_inventario(self):
    print(f"Inventario de {self.nombre}")
    for objeto in self.inventario:
        print(objeto)
```

print(f quiere decir que vamos a imprimir una cadena de caracteres, que va a contener unas que pueden ser reemplazadas y van entre llaves. Además, vamos a recorrer el array con la instrucción for (que ya estudiamos en la segunda unidad)

Ya estamos en disposición de crear nuestra clase mago y la de guerrero.

A continuación tienes el código de la clase Mago. Es necesario, como mínimo, redefinir las dos clases que estaban definidas como abstractas, la de estado y la de atacar.

Aunque podríamos añadir las que quisiésemos.

```
class Mago(Personaje):
    def __init__(self,nombre):
        super().__init__(nombre)
        self.vida=120
        self.inteligencia=100
        self.inventario=["Poción mágica","Libro Conjuros"]

    def estado(self):
        print(f"Clase Mago")
        super().estado()

    def atacar(self,objetivo):
        objetivo.vida-=self.inteligencia*0.6
        print(f"La vida actual del objetivo es {objetivo.vida}")
```

En el método atacar, lo que hace es quitar la vida al objetivo en proporción de la inteligencia del mago.

Podría añadir un método como saludar o lo que se nos ocurra. Esto no es algo cerrado.

Si has sido observador, verás que utilizamos la función super(), que utilizábamos para utilizar métodos de la clase padre.

Ahora toca la clase guerrero.

```
class Guerrero(Personaje):
    def __init__(self,nombre):
        super().__init__(nombre)
        self.vida=120
        self.fuerza=75
        self.inventario=["Poción de vida","Escudo","Espada"]

    def estado(self):
        print(f"Clase Guerrero")
        super().estado()

    def atacar(self,objetivo):
        objetivo.vida-=self.fuerza*0.8
        print(f"La vida actual del objetivo es {objetivo.vida}")
```

Creo que a estas alturas no haría falta explicar la clase Guerrero, ya que es parecida a la del Mago.

Ahora probamos todo para ver que funciona correctamente.

```
guerrero=Guerrero("He-Man")
mago=Mago("Merlin")

guerrero.estado()
mago.estado()

guerrero.ver_inventario()
mago.ver_inventario()

mago.atacar(guerrero)
guerrero.atacar(mago)
```

Creamos dos personajes, uno de tipo guerrero y otro de tipo mago.

Y a continuación mostramos el estado de cada uno y listamos su inventario.

Por último, se atacan entre los dos y mostramos el nivel de vida que le queda uno, lo cual hace el propio método atacar.

Éste es el resultado que obtendremos:

```
Clase Guerrero
El personaje He-Man tiene un nivel de 0
Clase Mago
El personaje Merlin tiene un nivel de 0
Inventario de He-Man
Poción de vida
Escudo
Espada
Inventario de Merlin
Poción mágica
Libro Conjuros
La vida actual del objetivo es 90.0
La vida actual del objetivo es 60.0
```

2.7 Ejercicios

A la hora de plantear ejercicios sobre herencia en Python, cualquier tema sería válido. Por lo tanto, podrías imaginar alguna clase que sea “la clase raíz” y de ahí crear un par de subclases.

Si no te propongo un ejemplo para que lo programes en Python.

Superclase o clase padre:

Animal

Para el constructor podríamos pasarle como parámetro la especie únicamente.

Propiedades: color – edad – especie

Métodos: moverse – comer – descripcion

Subclases:

Perro – Gato – Pájaro

A estas subclases añádeles las propiedades que tenga cada uno de manera particular y los métodos también. Por ejemplo, un pájaro puede volar y un perro no.

Acuérdate de usar pass si no vas a crear una clase en algún momento determinado.

Te dejo a tu elección que lo vayas complicando y completando todo lo que quieras.

También podrías crear la clase animal como una clase abstracta como hemos estudiado en el punto 2.6

3 Resumen

Puede que esta unidad te haya resultado un poco confusa o algo difícil, pero es la base de la Programación Orientada a Objetos. Y como ya te comenté, la hemos utilizado desde el principio del curso aunque no la conocieras.

La experiencia y los conocimientos se van adquiriendo con la práctica. Así que no te agobies demasiado.

Puedes repasar con calma y buscar más ejemplos y ejercicios en internet.

Estos son los conceptos clave que hemos aprendido.

- Las clases, con sus métodos y propiedades
- Las subclases y cómo heredar de una clase.
- Para qué utilizamos la sobreescritura de métodos.
- La función super para usar métodos de las clases superiores.
- Qué es el polimorfismo hablando de la POO y cómo lo utilizamos.
- La abstracción de clases y qué ventajas podemos aprovechar de ella.

Nos vemos en la próxima unidad, donde aprenderemos a trabajar con los ficheros y las bases de datos.



¡Enhorabuena!

Has finalizado el estudio de esta unidad



comunicación@bilib.es